Simple CTF: Write Up

Plataforma: TryHackMe



Máquina / Desafio:	Simple CTF (EasyCTF)
Dificuldade:	Fácil
Data de Resolução:	23/09/2026
Autor:	Alan Dias (alankk)

21 de outubro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **4**

4 Movimentação Lateral **6**

5 Escalada de Privilégios **6**

6 Conclusão **8**

7 Referências **8**

1 Visão Geral

A máquina **Simple CTF** (identificada na plataforma TryHackMe sob a denominação EasyCTF) é um ambiente prático voltado à identificação e exploração de vulnerabilidades em serviços de rede e aplicações web, associadas a falhas de configuração de privilégios em sistemas operacionais Linux. A superfície de ataque exposta compreende serviços de transferência de arquivos e uma aplicação web de gerenciamento de conteúdo. O percurso de intrusão abrange o reconhecimento externo e a enumeração de diretórios para a identificação de componentes vulneráveis, a obtenção de credenciais válidas para consolidação de acesso remoto inicial ao sistema operacional e, subsequentemente, a inspeção de diretivas de segurança locais para a condução de escalada vertical de privilégios até o comprometimento irrestrito como `root`.

2 Reconhecimento

O mapeamento da superfície de ataque iniciou-se por meio de uma varredura completa de portas TCP com o utilitário Nmap, visando detectar portas abertas, serviços ativos e versões operacionais no endereço de rede do alvo, conforme demonstrado no comando a seguir:

```
nmap -sC -p- -T4 <TARGET_IP>
```

Listing 1: Varredura abrangente de portas e serviços via Nmap

O relatório da varredura, ilustrado na Figura 1, retornou três portas TCP em estado aberto:

- **Porta 21/tcp (FTP):** Serviço `vsftpd 3.0.3` ativo com suporte a login anônimo (`Anonymous FTP login allowed`).
- **Porta 80/tcp (HTTP):** Servidor web Apache HTTP Server executando sobre sistema Ubuntu, exibindo página padrão de instalação e arquivos de diretivas no `robots.txt` com entradas restritas.
- **Porta 2222/tcp (SSH):** Serviço OpenSSH operando em porta alta não padronizada, identificável pelas assinaturas de chaves de host SSH (RSA, ECDSA e ED25519).

Prosseguiu-se para a fase de enumeração da aplicação web hospedada na porta 80. Para descobrir diretórios e caminhos não indexados na raiz do servidor web, utilizou-se a ferramenta de enumeração de força bruta Feroxbuster combinada à lista de termos comuns:

```
feroxbuster -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt
```

Listing 2: Enumeração de diretórios web com Feroxbuster

A Figura 2 detalha o retorno do utilitário, destacando a descoberta do diretório `/simple/` (código HTTP 301 redirecionando para `/simple/`), bem como subpastas de administração (`/simple/admin/`) e documentação (`/simple/doc/`).

Ao inspecionar a interface renderizada no diretório `/simple/`, identificou-se no rodapé da página que o sistema gerenciador de conteúdo em operação tratava-se do CMS Made Simple, na versão 2.2.8, conforme evidenciado na Figura 3.

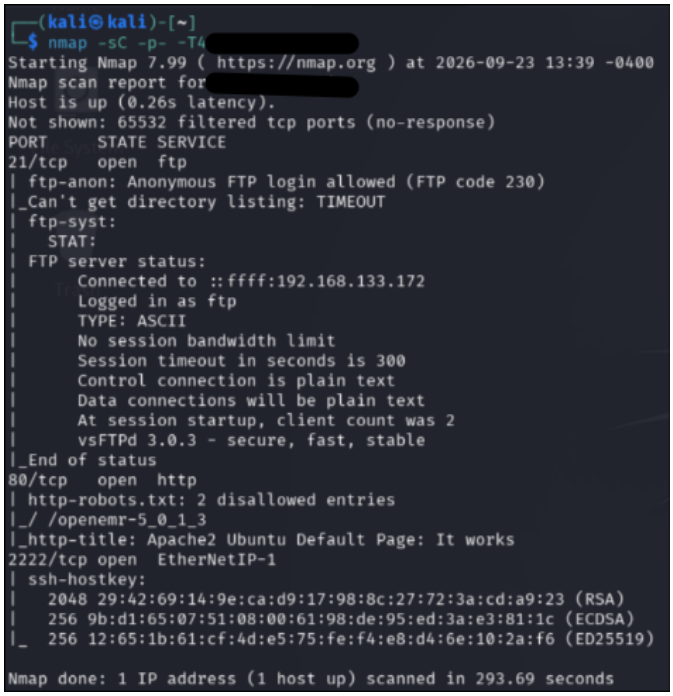


Figura 1: Resultado da enumeração de portas e serviços via Nmap.



Figura 2: Enumeração de diretórios e rotas ativas via Feroxbuster.



Figura 3: Versão do CMS Made Simple identificada no rodapé da aplicação.

3 Exploração

A partir da versão exata identificada (CMS Made Simple 2.2.8), consultaram-se as bases de vulnerabilidades públicas, localizando o boletim **CVE-2019-9053** [1]. A falha consiste em uma injeção de SQL cega baseada em tempo (Time-based Blind SQL Injection) no parâmetro `m1_idlist` do módulo `News`. A vulnerabilidade decorre da falta de sanitização adequada das entradas fornecidas pelo usuário antes de sua concatenação na consulta ao banco de dados MySQL, permitindo que agentes remotos não autenticados extraiam informações registro a registro por meio de atrasos controlados de tempo (`sleep`).

Para conduzir a extração automatizada de dados, empregou-se o script público de exploração em Python [2], direcionando a requisição à rota da aplicação:

```
python3 exploit_cms.py -u http://<TARGET_IP>/simple/
```

Listing 3: Execução do exploit para a falha CVE-2019-9053

O script inferiu os caracteres correspondentes ao sal criptográfico, usuário cadastrado e hash de senha, retornando os valores exibidos na Figura 4:

- **Salt extraído:** `<SALT>`
- **Nome de usuário:** `mitch`
- **Hash da senha:** `<HASH>`

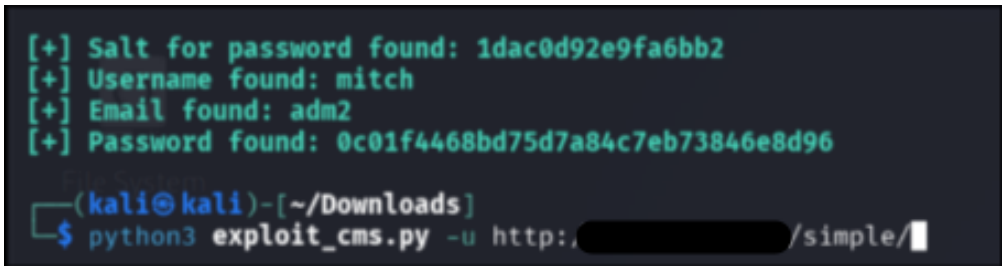


Figura 4: Extração de sal, usuário e hash criptográfico via CVE-2019-9053.

Para caracterizar o algoritmo empregado na derivação da credencial, utilizou-se o utilitário `hashid`, conforme apresentado na listagem a seguir:

```
hashid -m '<HASH>'
```

Listing 4: Identificação do formato de hash com hashid

A análise apontou compatibilidade primária com algoritmos da família MD5 (Hashcat modo 0, 20 ou variações correlatas), conforme exibido na Figura 5.

Tendo em vista a presença de sal associado ao CMS Made Simple, a estrutura de armazenamento adota o padrão `md5($salt.$pass)`, mapeado no Hashcat sob o modo `20`. Configurou-se um ataque baseado em dicionário utilizando a wordlist `rockyou.txt`:

```
hashcat -w 3 -a 0 -m 20 '<HASH>:<SALT>' /usr/share/wordlists/rockyou.txt
```

Listing 5: Quebra de hash criptográfico com Hashcat (modo 20)

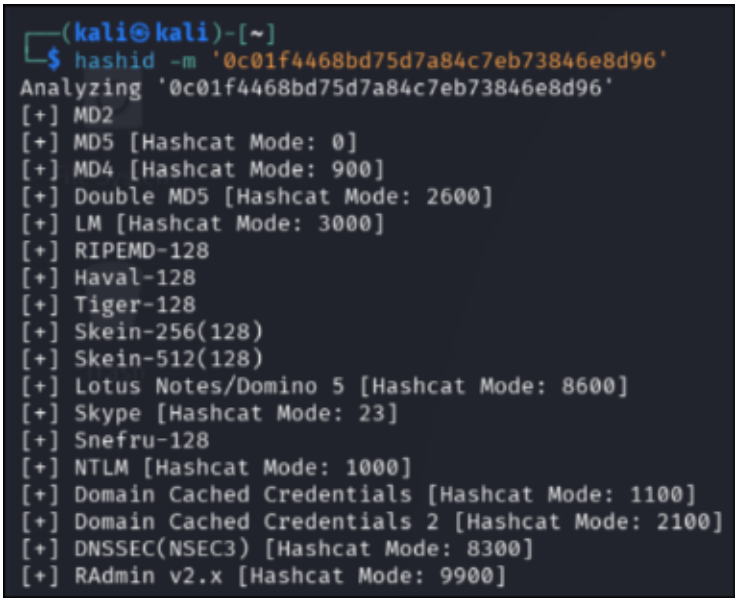


Figura 5: Resultado da identificação do hash via hashid.



Figura 6: Quebra do hash associado ao usuário mitch com Hashcat.

Conforme apresentado na Figura 6, o Hashcat concluiu o processamento com êxito, quebrando a credencial em menos de um segundo e revelando a senha em texto claro (`<PASSWORD>`).

Com o par de credenciais obtido (`mitch:<PASSWORD>`), correlacionou-se a presença do serviço OpenSSH em execução na porta não padrão 2222. Conectou-se ao servidor remoto autenticando-se diretamente via SSH:

```
ssh mitch@<TARGET_IP> -p 2222
cat user.txt
```

Listing 6: Acesso inicial via SSH e leitura da flag de usuário

O login foi estabelecido com sucesso, concedendo uma sessão interativa no sistema operacional Ubuntu 16.04. Na sequência, efetuou-se a leitura do artefato `user.txt` localizado no diretório pessoal do usuário, capturando a flag de usuário (`<FLAG>`), como demonstrado na Figura 7.

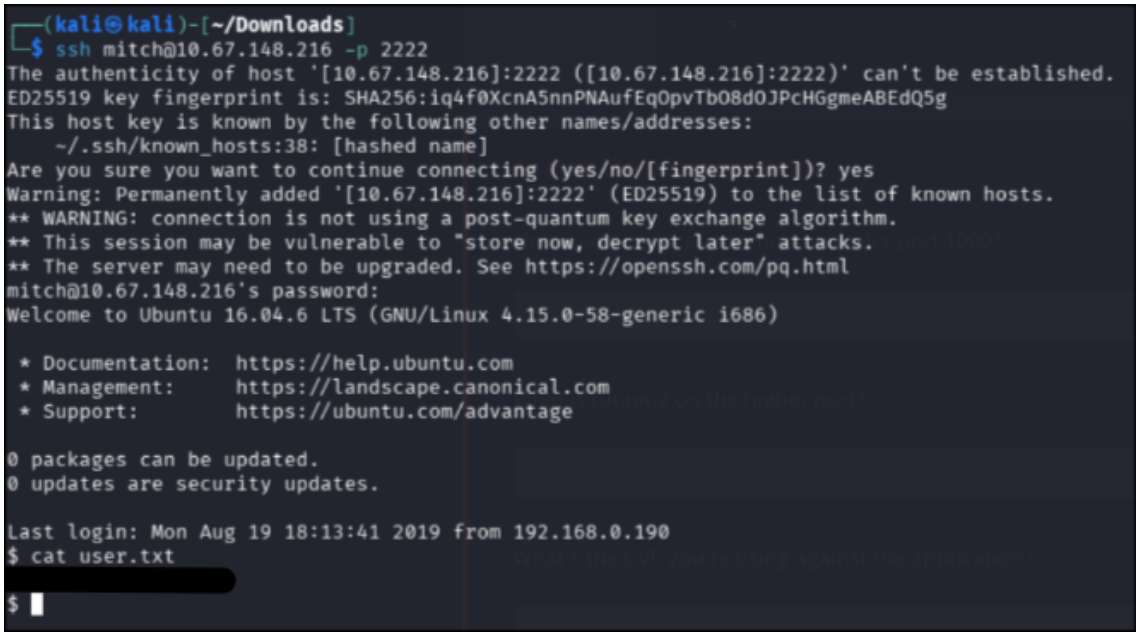


Figura 7: Sessão SSH estabelecida e captura da flag em user.txt.

4 Movimentação Lateral

Durante a inspeção do diretório `/home`, identificou-se a presença de uma segunda conta de usuário registrada no sistema, denominada `sunbath`. No entanto, a análise detalhada das permissões e privilégios locais da conta `mitch` evidenciou a existência direta de um vetor crítico de elevação vertical de privilégios para o superusuário `root`. Dessa forma, não foi necessária a realização de movimentação lateral ou transição horizontal de credenciais para alcançar o objetivo final.

5 Escalada de Privilégios

Consolidado o acesso como usuário de baixa prerrogativa, deu-se início à fase de enumeração local de privilégios. Executou-se a verificação de permissões delegadas no arquivo de configuração do Sudoers:

```
sudo -l
```

Listing 7: Verificação de regras delegadas no sudoers

A saída indicou que o usuário `mitch` possuía autorização para executar o utilitário `/usr/bin/vim` como superusuário sem a exigência de fornecer senha (`NOPASSWD: /usr/bin/vim`):

```
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
```

Listing 8: Regra identificada no sudoers

Conforme documentado no catálogo GTFOBins [3], editores de texto interativos executados sob o contexto do superusuário permitem invocar interpretadores de comandos ou scripts embarcados, quebrando o confinamento do aplicativo. Utilizou-se o recurso de execução de comandos do Vim para invocar o interpretador de comandos Bash através de sua interface Python:

```
sudo vim -c ':py import os; os.execl("/bin/bash", "bash")'
```

Listing 9: Elevação de privilégios através do Vim

A Figura 8 comprova a inspeção das diretivas do Sudo e a execução da linha de comando de elevação.

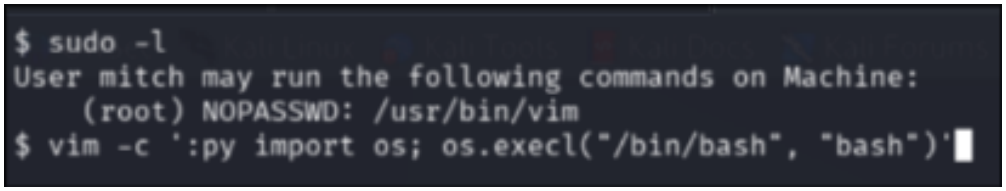


Figura 8: Enumeração de sudoers e execução de escape via Vim.

O comando gerou instantaneamente uma sessão de terminal sob o contexto do usuário com identificador UID 0 (`root`). Em seguida, navegou-se ao diretório do superusuário (`/root`) e obteve-se o conteúdo do artefato final `root.txt`:

```
root@Machine:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Machine:~# cat /root/root.txt
<FLAG>
```

Listing 10: Comprovação de acesso root e leitura da flag final

A Figura 9 atesta o comprometimento integral do sistema e a captura da flag de root (`<FLAG>`).



Figura 9: Prompt de root obtido e evidência da flag final.

6 Conclusão

A invasão da máquina Simple CTF sintetizou uma cadeia de comprometimento linear e representativa de falhas estruturais em ambientes corporativos. O vetor inicial baseou-se na exposição desnecessária de uma aplicação legada (CMS Made Simple 2.2.8) suscetível à injeção SQL cega (CVE-2019-9053), que permitiu a extração remota e posterior quebra de credenciais administrativas. Combinada à execução de serviços administrativos em portas não convencionais (SSH na 2222) e à configuração frouxa de permissões no utilitário Sudo sem senha para o editor Vim, a máquina permitiu a transição fluida do reconhecimento inicial ao controle irrestrito de privilégios como root.

7 Referências

Referências

[1] **National Vulnerability Database (NVD)**: CVE-2019-9053 - CMS Made Simple SQL Injection. Disponível em: https://nvd.nist.gov/vuln/detail/CVE-2019-9053

[2] **Repositório de Exploit Público**: Script de exploração em Python para CVE-2019-9053 (4nner). Disponível em: https://github.com/4nner/CVE-2019-9053/blob/master/exploit.py

[3] **GTFOBins**: Vim Privilege Escalation and Shell Spawning Techniques. Disponível em: https://gtfobins.github.io/gtfobins/vim/

[4] **TryHackMe**: Sala Simple CTF (EasyCTF). Disponível em: https://tryhackme.com/room/easyctf